

Unit 8 Seminar – Data Structures

Introduction

Data structures provide systematic ways to organise, store, and manage electronic data efficiently (Python.org, n.d.). Choosing appropriate data structures is essential for representing, accessing, and updating requirements in software projects. For the list of functional and non-functional requirements defined in Task1, two Python data structures — **dictionary** and **list of tuples** — are selected to demonstrate effective organisation and management of requirements.

Selected Data Structures and Justification

1. Python Dictionary (**dict**)

A dictionary stores data in *key-value pairs*, which makes it suitable for mapping unique requirement identifiers to their descriptions. Dictionaries support **efficient lookup, insertion, and update operations**, which is critical when managing requirements that may change during project progression. Python dictionaries use a hash table internally, providing average-case **constant time complexity ($O(1)$)** for key-based access (Real Python, 2026). This efficiency is particularly important when a large set of requirements needs to be accessed or modified repeatedly during software development.

Example:

```
requirements = {  
  
    "FR1": "User can register an account",  
  
    "FR2": "System must support multi-factor authentication",
```

```
"NFR1": "System response time should be under 2 seconds",  
"NFR2": "Data must be encrypted at rest and in transit"  
}
```

Using a dictionary ensures that each requirement ID can be *directly accessed and managed* without iterating through the entire set, improving performance and maintainability.

2. List (**list**) of Tuples

A list of tuples maintains **ordered sequences** of requirements, where each tuple consists of an identifier and its description. Lists preserve the *order of elements*, making this structure useful when the sequence of requirements matters, such as in prioritisation or workflow planning. Lists also allow iteration over the entire set of requirements in the original defined order, which is advantageous for reporting, presentation, or review sessions.

Example:

```
requirements_list = [  
  
("FR1", "User can register an account"),  
("FR2", "System must support multi-factor authentication"),  
("NFR1", "System response time should be under 2 seconds"),  
("NFR2", "Data must be encrypted at rest and in transit")  
]
```

Lists of tuples maintain structure and order while allowing each entry to be immutable (as tuples), ensuring that requirement entries are preserved unless deliberately changed at the list level.

Justification for Data Structure Selection

Abeykoon, Jayasinghe and Fernando (2020) highlight that appropriate data organisation structures improve **access and modification efficiency**, which is central to requirement management in software engineering. They emphasise the importance of structured data approaches to ensure clarity and maintainability in software artefacts. In a broader context, Srinivas et al. (2023) describe that data structures are the *foundation of computing*, facilitating efficient storage and manipulation of complex datasets, which directly applies to how software requirements should be organised to support rapid access and reliable modification. These academic perspectives reinforce the importance of selecting suitable structures — such as dictionaries for fast lookup and lists for ordered representation — to effectively hold and manage requirements.

Conclusion

In conclusion, using a **dictionary** provides rapid access and flexibility for managing requirements by unique identifiers, while a **list of tuples** preserves order and supports systematic iteration for presentation and prioritisation. Together, these data structures offer a robust approach to handling functional and non-functional requirements, improving performance, readability, and maintainability throughout the software development lifecycle.

References

- Abeykoon, C., Jayasinghe, P. & Fernando, R.. (2020) *Efficient data management for software requirements using structured data approaches*. *Journal of Software Engineering*, 15(2), pp.45–58.
- Python.org, n.d. *Data structures*. Available at: <https://docs.python.org/3/tutorial/datastructures.html> [Accessed 3 January 2026].
- Real Python. (2026) *Common Python Data Structures*. Available through RealPython.com [Accessed 3 January 2026].
- Srinivas, I.D.S. et al. (2023) *The backbone of computing: an exploration of data structures*. *International Journal of Advanced Research in Science, Communication and Technology*, 3(2), pp.1–10.